

SIMATS ENGINEERING

ASSESSMENT TOOL 1

COURSE NAME: OBJECT ORIENTED ANALYSIS AND DESIGN
COURSE CODE: CSA1110

COURSE OUTCOME COVERED

CO1: Apply object-oriented concepts and software development life cycle models to design structured and modular software systems. (BL2, BL3)

Assessment Tool Weightage: 60%

Dr Manimaran

QUESTIONS: 12

Total Marks:60

Annexure A

CONCEPT MAPPING

Code of Conduct:

I ___Delisha A K(192511307)_____ (Name / Reg No) certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment.

I understand that any violation of academic integrity rules will result in disciplinary action.

Signature of the student:

Question 1

Construct a Concept Map for the Software Development Life Cycle phase mapping:

Given: Requirement Analysis → [Missing 1] → Coding → Testing → [Missing 2] → Maintenance

Missing Concepts:

- 1. Design
- 2. Deployment

Explanation:

The Software Development Life Cycle (SDLC) represents a structured framework tracking a system's evolution from conception to decommissioning. The first missing concept, Design, converts user needs established during Requirement Analysis into comprehensive systemic blueprints. This phase is sub-divided into High-Level Design (HLD), which focuses on system architecture and component interactions, and Low-Level Design (LLD), which establishes exact class structures, database schemas, and data dictionary formats.

The second missing phase, Deployment, happens immediately post-Testing. It encompasses the structured delivery of verified, integrated software builds into production environments. Modern deployment models depend heavily on automated CI/CD pipelines, configuration management strategies, and staging validation routines to guarantee minimal operational friction for end users.

Systemic Analysis:

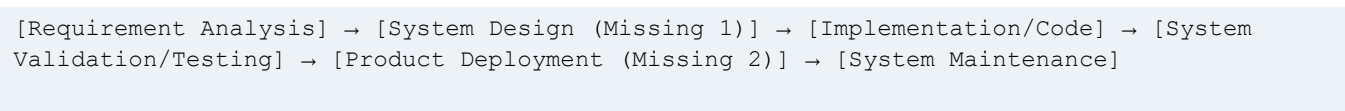
From an engineering perspective, this sequential lifecycle acts as a systematic filter to prevent defects from leaking downstream. Establishing the Design phase explicitly structures architectural boundaries before any source code is generated, reducing re-engineering overhead. Validating systems via an independent Testing tier before Deployment guarantees system stability and protects the core code base.

Industrial Case Analysis: In large-scale enterprise banking upgrades, omitting a formalized Design gate frequently results in architectural mismatches during integration. By enforcing strict entry and exit criteria for every phase, development teams maintain traceability between initial user requirements and the deployed runtime environment.

Structural Concept Map:



UML System Structural Blueprint Architecture Mapping Layout:



Question 2

Construct a Concept Map for foundational Object-Oriented Design rules:

Given: Object-Oriented Principles → Abstraction, [Missing 1], Inheritance, [Missing 2]
Missing Concepts:

1. Encapsulation
2. Polymorphism

Explanation:

The core pillars of Object-Oriented Analysis and Design provide the foundation for building maintainable, modular software systems. Encapsulation wraps state data variables and operational behaviors inside a single class, enforcing access control through visibility modifiers (public, private, protected) to protect data integrity.

Polymorphism allows a single interface to represent multiple distinct underlying behaviors. Through runtime or dynamic binding, the execution environment dynamically maps generalized method invocations to specific subclass implementations, enabling highly adaptable system architectures.

Systemic Analysis:

Enforcing structural Encapsulation minimizes cross-subsystem coupling, ensuring that updates to internal class logic do not break external dependencies. Simultaneously, leveraging Inheritance alongside open Polymorphism enables software systems to scale dynamic components smoothly without altering legacy base source configurations.

Industrial Case Analysis: In payment gateway routing infrastructures, polymorphism allows developers to add new payment processors (e.g., PayPal, Stripe) by extending a base `PaymentProcessor` class. The core checkout engine invokes `.processTransaction()` uniformly, removing the need for complex, unmaintainable nested conditional statements.

Structural Concept Map:

```
[Object-Oriented Principles]
├─ [Abstraction] (Hides complex internal details)
├─ [Encapsulation] (MISSING 1) → Restricts Direct Access to State Data
├─ [Inheritance] (Reuses code structure across hierarchies)
└─ [Polymorphism] (MISSING 2) → Overloading (Static) & Overriding (Dynamic)
```

UML System Structural Blueprint Architecture Mapping Layout:

```
[Class Box Boundary]
├─ Private State Data (- balance, - pin) ◀ Encapsulated Gate Layer
└─ Public Methods (+ withdraw(), + login())

[Dynamic Polymorphic Dispatch]
└─ BaseProcessor.process() → Triggers Dynamic Routing to [PayPalImpl] or [StripeImpl]
```

Question 3

Construct a Concept Map mapping runtime element characteristics:

Given: Class → Blueprint, Object → Instance, Method → [Missing 1], Attribute → [Missing 2]

Missing Concepts:

1. Behavior
2. State

Explanation:

Object-oriented programming splits system components into layout descriptions and runtime instances. Methods define the operational Behavior of an instantiated object, acting as the dynamic logic that updates internal states and handles communication between objects.

Attributes represent variable properties that hold the in-memory State of an object throughout its lifecycle. The state of an object reflects its current internal configuration, which can change as methods run, while the overall class layout remains constant.

Systemic Analysis:

Decoupling system operations (Behavior) from data retention layers (State) allows developers to model real-world business domains accurately. System stability depends on protecting state integrity. Restricting direct state modifications to authorized class methods prevents data corruption and makes debugging much more straightforward.

Industrial Case Analysis: Consider a flight reservation system. The 'Flight' object contains attributes like availableSeats (State). If multiple external threads modify availableSeats directly without using a synchronized 'bookSeat()' method (Behavior), the system risks double-booking errors, highlighting the need for controlled state manipulation.

Structural Concept Map:

```
[Class] → (defines static operational blueprint) → Contains structural definitions
[Object] → (creates real-time heap instance) → Allocated in-memory runtime execution
[Method] → (exhibits functional action) → [Behavior] (MISSING 1: Functional algorithms)
[Attribute] → (maintains variable property) → [State] (MISSING 2: Property values)
```

UML System Structural Blueprint Architecture Mapping Layout:

```
[delishaAccount : Bank] ← Instantiated Runtime Object Instance from Class Blueprint
├ - balance: 50000.00
├ - status: Active ← [State Value Context]
├ + deposit(amount)
└ + checkStatus() ← [Behavior Operational Interface]
```

Question 4

A Concept Map Shows Quality Characteristics Mapping:

Given: Software Quality → Reliability, [Missing Concept], Maintainability

Missing Concept:

1. Usability

Explanation:

Software product excellence relies on key quality characteristics defined by standard software engineering benchmarks (such as ISO/IEC 25010). Usability measures how easily and efficiently users can interact with the system, focusing on learnability, task efficiency, and accessibility.

Reliability ensures accurate, fault-tolerant execution under defined operations over specific periods. Maintainability measures how easily engineering teams can update, fix, or enhance the codebase without breaking existing features.

Systemic Analysis:

High core reliability paired with clean maintainability can still result in software failure if usability standards are ignored. Ensuring an intuitive user interface layout minimizes operational friction and reduces downstream training overhead, making usability critical to user adoption.

Industrial Case Analysis: Medical charting systems often boast high availability (Reliability) and clean backend code (Maintainability). However, if the user interface is overly complex (poor Usability), data entry delays can severely hurt clinical workflows, showing that non-functional requirements are deeply interconnected.

Structural Concept Map:

```
[Software Quality Metrics]
├─ [Reliability] → Fault Tolerance & Mean Time Between Failures (MTBF)
├─ [Usability] (MISSING CONCEPT) → Interface Design, Learnability & Task Efficiency
└─ [Maintainability] → Modular Code Base, Extensibility & Reduced Complexity
```

UML System Structural Blueprint Architecture Mapping Layout:

```
[Software Quality Matrix Architecture]
├─ [Execution Focus Node] → Handles Runtime Reliability
├─ [User Adoption Node] → Handles UI/UX Usability [Missing Concept]
└─ [Code Lifecycle Node] → Handles Evolution Maintainability
```

Question 5

UML Diagram Classification Mapping:

Given: UML → Structural Diagrams (→ [Missing 1]), Behavioral Diagrams (→ Activity Diagram), Interaction Diagrams (→ [Missing 2])

Missing Concepts:

1. Object Diagram
2. Communication Diagram

Explanation:

The Unified Modeling Language (UML) categorizes architectural models based on perspective. Object Diagrams fit under Structural Diagrams, capturing static snapshot relations between instantiated system objects at a specific point in execution.

Communication Diagrams belong to Interaction sub-categories, focusing on organized structural message transfers across interconnected nodes. While Sequence Diagrams display message exchanges along a linear timeline, Communication Diagrams highlight the structural relationships between objects directly.

Systemic Analysis:

Structural diagrams define the static layout of system components, whereas interaction diagrams capture dynamic system workflows over time. Object diagrams validate class models by mapping test scenarios, while communication diagrams help track message routing paths to find performance bottlenecks in system logic.

Industrial Case Analysis: When refactoring legacy distributed systems, developers use Object Diagrams to visualize runtime memory states. They then use Communication Diagrams to trace network calls across microservices, ensuring that inter-service dependencies match the planned architectural boundaries.

Structural Concept Map:

```
[UML Diagrams Hierarchy]
├─ [Structural] → Class Diagram & [Object Diagram] (MISSING 1: Static Snapshot)
├─ [Behavioral] → Activity Diagram & State Machine Models
└─ [Interaction] → Sequence Diagram & [Communication Diagram] (MISSING 2: Structural Link Focus)
```

UML System Structural Blueprint Architecture Mapping Layout:

```
[UML Classification Taxonomy]
[UML Diagrams Level 0]
  → [Structural Tier] → Enforces [Object Diagram]
  → [Behavioral Tier] → Enforces [Activity Diagram]
  → [Interaction Tier] → Enforces [Communication Diagram]
```

Question 6

Given Lifecycle Pipeline Breakdown Scenario:

Given: Requirements → Design → Coding (X) → Testing (X) → Deployment (X)

Explain the failure and provide structural engineering solutions.

Root Cause Analysis:

Critical execution breakdown inside the Coding phase (Implementation Phase Failure).

Explanation:

The process breakdown occurs within the Coding execution environment. Although system requirements and architectural design specifications were successfully validated, syntax flaws, logical errors, or uncompiled modules during code generation broke the software pipeline, blocking validation and release efforts.

When compiling fails or major logic errors appear during coding, the codebase cannot produce stable executables, completely halting downstream validation stages.

Systemic Analysis:

Syntax and compilation defects early in the pipeline completely stall downstream Quality Assurance operations. When builds fail during the Coding phase, testers cannot execute functional cases, which delays project timelines and introduces bugs due to rushed patches later on.

Industrial Case Analysis: In automated aerospace software engineering, a single syntax or type mismatch during coding can ground entire integration builds. Resolving these errors early prevents expensive code rewrites during integration testing.

Structural Concept Map:

```
[Requirements: Status OK] → [Design: Status OK] → [Coding: STATUS CRASH] (Blocks downstream pipeline builds)
├─ [Testing: STATUS BLOCKED]
└─ [Deployment: STATUS BLOCKED]
```

UML System Structural Blueprint Architecture Mapping Layout:

```
[Pipeline Interruption State Model]
  [Requirements: Passed] → [Design: Passed] → [Coding: Phase Crash Site] ⊗ [Testing / Deployment Aborted]
  [Strategic Remediation Blueprint]
    1. Implement strict automated syntax linting and mandatory compilation checks prior to assembly.
    2. Conduct structured, peer-driven code inspections matching verified design blueprints.
    3. Use Continuous Integration (CI) tools to run unit tests automatically on every code commit.
```

Question 7

Construct a Concept Map for sequential system development lifecycle steps:

Given: Problem Identification → [Missing 1] → Design → Implementation → [Missing 2]

Missing Concepts:

- 1. Requirement Analysis
- 2. Testing

Explanation:

Engineering lifecycles rely on structural feedback loops to ensure software quality. Requirement Analysis gathers and documents client needs immediately after Problem Identification. This step produces the Software Requirements Specification (SRS), which serves as the legal and technical foundation for the project.

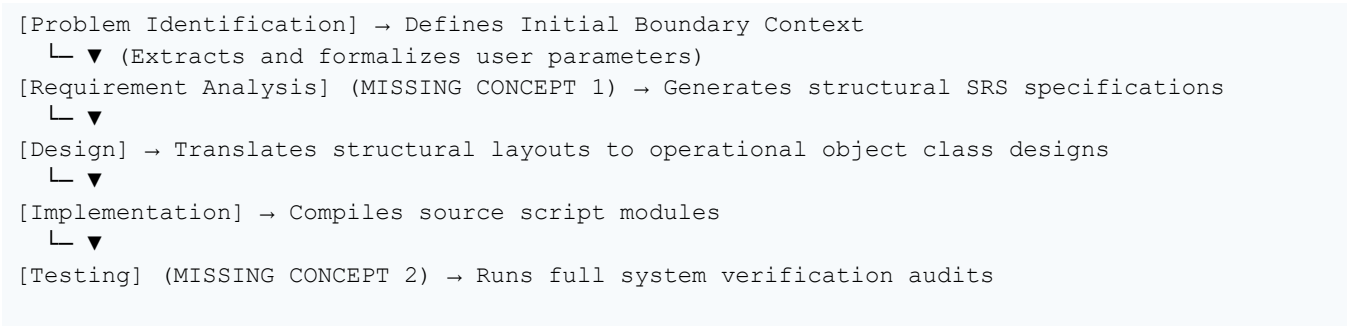
Following physical source code Implementation, the Testing phase executes validation cases to ensure the software matches the baseline engineering criteria. Testing spans unit, integration, system, and user acceptance testing (UAT) to catch defects before release.

Systemic Analysis:

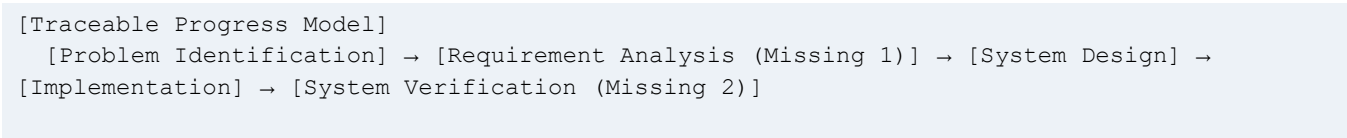
Skipping Requirement Analysis leads to scope creep and incorrect design configurations. Similarly, omitting organized Testing risks delivering unstable code to production. Including both steps ensures the software satisfies user requirements and meets stability standards.

Industrial Case Analysis: Government IT project overruns are frequently traced back to poor requirement tracking. When requirements are loosely defined, implementation becomes an unstable target, leading to high failure rates during late-stage validation.

Structural Concept Map:



UML System Structural Blueprint Architecture Mapping Layout:



Question 8

Construct a Concept Map for Object-Oriented paradigm attributes:

Given: Object-Oriented Features → Class, [Missing 1], Message Passing, [Missing 2]

Missing Concepts:

1. Object
2. Dynamic Binding

Explanation:

These components orchestrate execution behaviors within object-oriented software systems. Objects are active instances of classes that hold runtime state data allocated in memory.

Dynamic Binding defers method-to-call resolution until runtime, letting the system invoke polymorphically overridden behaviors on the fly. This contrasts with static binding, where method calls are resolved early during compilation.

Systemic Analysis:

Classes act as static design layouts, but system logic only executes when objects are allocated in memory. Using Dynamic Binding alongside Message Passing removes the need for hardcoded switch statements, allowing applications to load and swap target objects without breaking existing interfaces.

Industrial Case Analysis: Modern operating system device drivers use dynamic binding extensively. The kernel sends a generic 'read()' message, and dynamic binding routes it to the specific driver object (e.g., USB, SSD), allowing new hardware to be added without recompiling the core OS.

Structural Concept Map:

```
[Object-Oriented Features Mapping]
├─ [Class] → Establishes Static Structural Form Definitions
├─ [Object] (MISSING CONCEPT 1) → Instantiates Live Runtime Heap Memory
├─ [Message Passing] → Handles Dynamic Inter-object Communications
└─ [Dynamic Binding] (MISSING CONCEPT 2) → Executes Adaptive Runtime Polymorphic
    Decisions
```

UML System Structural Blueprint Architecture Mapping Layout:

```
[Runtime Polymorphic Routing Logic]
[Client Source Node] → Dispatches message: shape.draw()
└─ ▼
    [Dynamic Binding Registry Engine] → Evaluates concrete metadata structure at runtime
        ├── If type is Circle → Resolves execution pointer path to: Circle.draw()
        └── If type is Square → Resolves execution pointer path to: Square.draw()
```

Question 9

Construct a Concept Map for systemic Software Engineering process models:

Given: Software Development Models → Waterfall, [Missing 1], Spiral, [Missing 2]

Missing Concepts:

1. Incremental Model
2. Agile Model

Explanation:

Software engineering process models define the workflows used to manage system development. The Incremental Model builds and delivers the software in functional chunks, with each release adding new features onto a verified base.

The Agile Model focuses on iterative refinement, short development cycles (sprints), and continuous user feedback. Agile prioritizes adaptable planning and working software over rigid documentation, helping teams respond quickly to changing project environments.

Systemic Analysis:

While Waterfall works well for projects with fixed, stable requirements, its rigid structure makes adapting to changes difficult. Integrating Incremental and Agile methodologies helps manage project risks by providing early working builds, ensuring the system can evolve along with changing client needs.

Industrial Case Analysis: Cloud-based SaaS applications rely almost exclusively on Agile deployment models to launch features weekly. In contrast, safety-critical systems like flight control software still use structured Waterfall or Spiral variants due to strict validation and regulatory requirements.

Structural Concept Map:

```
[Software Development Process Models]
├─ [Waterfall] → Sequential, Linear Framework & Rigid Phase Boundaries
├─ [Incremental Model] (MISSING 1) → Delivers Staged Functional Component Blocks
├─ [Spiral] → Risk-Driven Analysis Architecture utilizing Radial Iteration Steps
└─ [Agile Model] (MISSING 2) → Continuous Sprints, High Adaptability & Iterative Feedback
    Loops
```

UML System Structural Blueprint Architecture Mapping Layout:

```
[Development Model Framework Paradigms]
├─ [Waterfall]: [Req] → [Des] → [Code] → [Test] (Linear Drop)
├─ [Incremental]: Build A → Build B → Modular System Expansion Steps
├─ [Spiral]: Radial Rotations centered directly around Continuous Risk Evaluations
└─ [Agile]: Highly Interactive Collaborative Sprints focused heavily on Swift Adaptations
```

Question 10

Construct a Concept Map for Class Diagram structure composition:

Given: Class Diagram → Class, [Missing 1], Association, [Missing 2]

Missing Concepts:

1. Attribute
2. Method

Explanation:

UML Class Diagrams map out the static structure of an object-oriented system. Classes contain Attributes, which define the structural data properties, and Methods, which implement the object's operational logic.

Association lines represent static relationships and structural links between these classes. These connections can be further refined into Aggregation (whole-part relationships) or Composition (strong ownership relationships where parts cannot exist independently).

Systemic Analysis:

Attributes and methods define the internal structure of a class, while associations map external system connections. Clearly documenting these details in UML models simplifies the transition from design to source code, ensuring clean system maintenance down the line.

Industrial Case Analysis: In enterprise ERP engineering, clear class diagrams prevent entity relationship errors. Mapping multiplicities (e.g., one 'Order' to many 'LineItems') directly guides database foreign key constraints during generation.

Structural Concept Map:

```
[Class Diagram Components]
├─ [Class Box Structural Node]
│   ├── [Attribute] (MISSING 1) → Defines System Data State Properties
│   └── [Method] (MISSING 2) → Defines Executable Class Operations
└─ [Association Links Relationship Edges] → Handles Mappings like Generalization & Dependency
```

UML System Structural Blueprint Architecture Mapping Layout:

```
[UML Standard Class Block Notation]
+-----+
|               CourseClass               |
+-----+
| - courseCode : String           [Attribute 1] |
+-----+
| + getDetails() : Void           [Method 2]    |
+-----+
| 1..* (Establishes Class Association Linkage Multiplicity)
| ▼
+-----+
|               StudentClass              |
+-----+
```

Question 11

Construct a Concept Map for fundamental Software Engineering goals:

Given: Software Engineering Principles → Modularity, [Missing 1], Reusability, [Missing 2]

Missing Concepts:

1. Maintainability
2. Scalability

Explanation:

Core software engineering principles guide the design of reliable enterprise software systems. Maintainability ensures that engineering teams can quickly patch bugs or update system logic without risking breaking regressions.

Scalability allows the software architecture to handle larger workloads, increased user traffic, and growing transaction volumes efficiently, using horizontal scale strategies without needing a total system redesign.

Systemic Analysis:

Building systems with high modularity and reusability directly improves maintainability and scalability. Well-isolated modules make it easy to deploy updates without causing regression bugs, helping long-term software applications scale smoothly.

Industrial Case Analysis: Microservice migrations at companies like Netflix highlight these goals. Splitting large legacy systems into independent modules lets specific high-traffic services scale autonomously on cloud infrastructure.

Structural Concept Map:

```
[Software Engineering Blueprint Principles]
├─ [Modularity] → Separates Cohesive Subsystems neatly
├─ [Maintainability] (MISSING 1) → Lowers overall long-term engineering operational
expenses
├─ [Reusability] → Leverages shared structural components widely
└─ [Scalability] (MISSING 2) → Smoothly handles growing high-volume execution capacities
```

UML System Structural Blueprint Architecture Mapping Layout:

```
[Engineering Goals Architectural Viewport]
[Target: High Architectural Quality Baseline]
  → Sustained directly via [Maintainable Infrastructure Frameworks]
  → Expanded dynamically via [Scalable Distributed Architecture Models]
    ▲ Combined core goals mapping directly back to underlying [Discovered Design
Principles]
```

Question 12

Given Integration Lifecycle Pipeline Breakdown Scenario:

Given: Requirement Analysis → System Design → Coding → Integration (X) → Deployment (X)
Explain the breakdown and provide structural engineering solutions.

Root Cause Analysis:

Critical execution breakdown inside the Integration phase (Subsystem Interface Mismatch Failure) .

Explanation:

The system pipeline breaks during the Integration phase. While individual modules were coded successfully according to design specifications, they failed to communicate correctly when combined.

This issue usually points to mismatched API interfaces, conflicting data serialization formats (e.g., one module sending XML while another expects JSON), or race conditions within the shared environment.

Systemic Analysis:

Integration failures slow down deployment timelines and point to isolated unit coding without interface verification. When system boundaries do not match up, data gets corrupted across endpoints, which completely halts release schedules.

Industrial Case Analysis: Healthcare software upgrades often run into integration bottlenecks when linking electronic record modules with external billing gateways. Enforcing strict API compliance validation before system assembly prevents these mid-stage delivery failures.

Structural Concept Map:

```
[Analysis: OK] → [Design: OK] → [Coding: OK] → [Integration: STATE CRASH] → [Deployment:
BLOCKED OUT]
```

UML System Structural Blueprint Architecture Mapping Layout:

```
[Component Subsystem Boundary Interface Breakdown Model]
  [Subsystem Module A] → Outputs: JSON Object Structure Specification Format
    |
    ▼ × [CRASH SITE INTERACTION HIGHWAY: Mismatched Runtime Data Protocols]
    ▲
  [Subsystem Module B] → Demands Input: Strict XML Structured Layout Parameters
  [Strategic Remediation Blueprint]
  1. Define explicit, immutable inter-module API interface contracts before coding starts.
  2. Set up automated continuous integration testing pipelines to validate combined
  subsystem builds.
  3. Use stub and mock frameworks early on to catch boundary mismatches before system
  runtime assembly.
```

CONCEPT MAPPING

RUBRICS FOR CALCULATION

CONCEPT MAPPING

Criteria	Weightage	Level 4 (Exemplary)	Level 3 (Proficient)	Level 2 (Developing)	Level 1 (Beginning)
Concept Identification	25%	Identifies all key concepts from literature accurately and comprehensively	Identifies most key concepts with minor omissions	Identifies limited concepts; some are irrelevant or missing	Fails to identify essential concepts
Linkages	25%	Establishes clear, meaningful, and accurate relationships between concepts	Shows correct relationships with minor gaps	Relationships are weak, unclear, or partially incorrect	No meaningful relationships established
Organization	20%	Concepts are well-structured hierarchically with logical flow and grouping	Mostly organized with minor structural issues	Some organization but lacks clear hierarchy	No logical organization or structure
Integration of Literature	20%	Integrates multiple studies effectively to show connections and comparisons	Shows some integration of literature	Limited integration, mostly isolated concepts	No integration of literature evidence
Clarity	10%	Concept map is clear, neat, visually structured, and easy to interpret	Generally clear with minor readability issues	Clarity is reduced, difficult to interpret in parts	Poorly presented and hard to understand